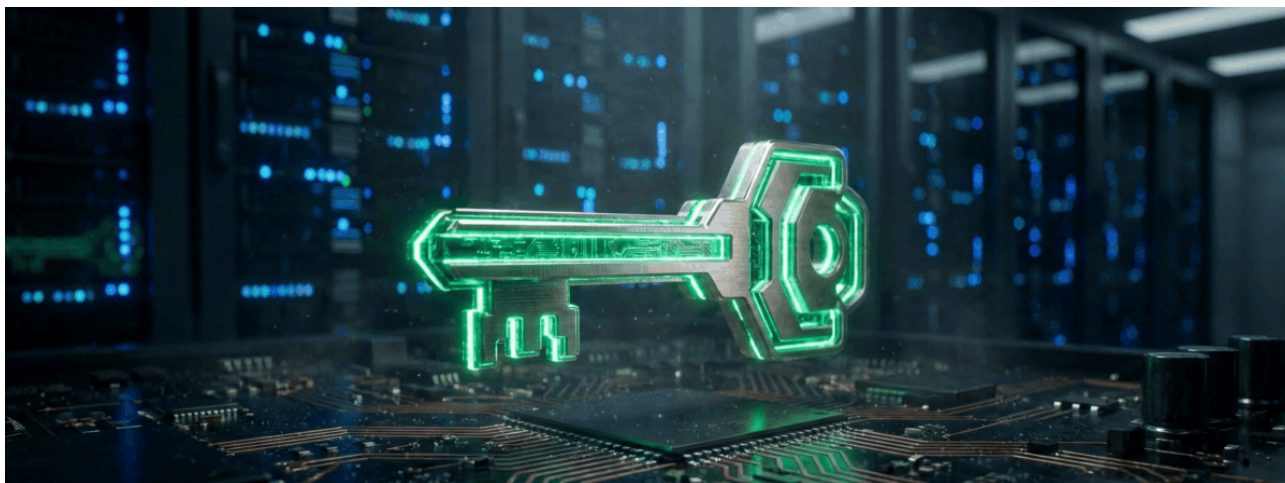


SSH ключи: Полный справочник для специалиста по информационной безопасности

 redsec.by/article/ssh-klyuchi-polnyj-spravochnik-dlya-specialista-po-informacionnoj-bezopasnosti

Red Teams | 21 января 2026



В современной инфраструктуре протокол SSH (Secure Shell) является де-факто стандартом для удаленного администрирования. Однако, как показывает практика аудитов безопасности, именно SSH часто становится «золотым ключом» для злоумышленников. В отличие от компрометации пароля, которая часто ограничена одной учетной записью, скомпрометированный SSH-ключ может предоставить атакующему мгновенный, скрытый и привилегированный доступ к десяткам, а иногда и тысячам серверов организации.

Для инженера по информационной безопасности понимание механики работы SSH — это не просто навык настройки подключения. Это глубокое знание криптографических примитивов, векторов атак (от MITM до Supply Chain) и методов защиты, соответствующих строгим стандартам, таким как ОАЦ (Оперативно-аналитический центр) и ТЗИ. В этом руководстве мы разберем анатомию SSH-аутентификации, актуальные уязвимости 2024-2025 годов и построим эшелонированную оборону.

1. Как работают SSH ключи: Криптографический фундамент

Безопасность SSH базируется на асимметричной криптографии. Это математическая концепция, использующая пару ключей:

- **Приватный ключ (Private Key):** Секретный компонент, который должен храниться исключительно на стороне клиента и никогда не покидать защищенный периметр устройства.

- **Публичный ключ (Public Key):** Открытый компонент, который размещается на целевых серверах.

Главное криптографическое свойство этой пары заключается в невозможности математически вычислить приватный ключ, имея на руках только публичный. Это позволяет безопасно распространять публичные ключи по всей инфраструктуре. Важно понимать: при SSH-аутентификации не происходит шифрования пароля. Вместо этого используется механизм **цифровой подписи**.

Эволюция алгоритмов: От RSA к Ed25519

Выбор алгоритма генерации ключей критически влияет на производительность и устойчивость к взлому.

- **RSA (Rivest–Shamir–Adleman):** Долгое время являлся индустриальным стандартом. Его надежность зависит от сложности факторизации больших простых чисел.
 - *Статус:* RSA 2048-bit официально считается устаревшим и слабым для долгосрочного хранения данных.
 - *Рекомендация:* Если вы вынуждены использовать RSA для поддержки legacy-систем, минимально допустимая длина ключа — **4096 бит**. Однако стоит помнить, что операции с такими ключами (особенно на мобильных устройствах или IoT) медленны.
- **Ed25519 (Edwards-curve Digital Signature Algorithm):** Современный «золотой стандарт», основанный на эллиптических кривых (EdDSA).
 - *Преимущества:* Ключ длиной всего 256 бит криптографически эквивалентен RSA 4096 бит, но операции подписи и проверки происходят на порядки быстрее.
 - *Безопасность:* Математический дизайн кривой делает алгоритм устойчивым к атакам по сторонним каналам (side-channel attacks). Поддерживается в OpenSSH начиная с версии 6.5 (2014 год).
- **ECDSA:** Предшественник Ed25519 на эллиптических кривых. Работает быстрее RSA, но имеет более сомнительную репутацию из-за зависимости от качественного генератора случайных чисел (RNG).
- **DSA:** Полностью устарел (Deprecated). Современные дистрибутивы Linux и версии OpenSSH по умолчанию блокируют доступ с такими ключами.

Вердикт для специалиста: Для всех новых развертываний используйте **Ed25519**. Для старых систем планируйте миграцию с RSA 2048 на более стойкие алгоритмы.

2. Анатомия SSH аутентификации

Многие администраторы считают, что SSH просто «сравнивает файлы». На самом деле процесс представляет собой строгий криптографический протокол «Challenge-Response» (Вызов-Ответ).

- **Инициализация:** Клиент генерирует пару ключей (ssh-keygen). Приватный остается в ~/.ssh/, публичный копируется на сервер в файл ~/.ssh/authorized_keys.
- **Запрос:** При попытке подключения клиент отправляет серверу имя пользователя и ID публичного ключа, который он хочет использовать.
- **Проверка наличия:** Сервер ищет этот ключ в своем списке доверенных. Если находит — генерирует «Challenge».
- **Вызов (Challenge):** Сервер создает случайный набор байтов (nonce), шифрует его публичным ключом клиента и отправляет обратно.
- **Подпись:** Клиент, используя свой **приватный ключ**, расшифровывает вызов, добавляет к нему идентификатор сессии (Session ID) и создает цифровую подпись.
- **Верификация:** Сервер проверяет полученную подпись с помощью публичного ключа. Если математика сходится — клиент доказал владение секретом, не передавая сам секрет по сети.

После успешной аутентификации происходит согласование симметричного ключа (обычно AES или ChaCha20), который шифрует весь дальнейший трафик сессии.

3. Базовая гигиена и защита ключей

Безопасность начинается на локальной машине администратора. Если исходный ключ скомпрометирован, никакие настройки сервера не спасут инфраструктуру.

Правильная генерация

Используйте флаги для повышения стойкости ключа к брутфорсу (в случае кражи файла с паролем).

```
# Рекомендуемый стандарт (Ed25519)
# -o: использовать новый формат OpenSSH (более защищенный)
# -a 256: увеличить количество раундов KDF (функции деривации ключа) для защиты парс
ssh-keygen -o -a 256 -t ed25519 -f ~/.ssh/id_ed25519 -C "admin@company-$(date +%Y-%m-%d)"

# Для устаревших систем (RSA 4096)
ssh-keygen -o -a 256 -t rsa -b 4096 -f ~/.ssh/id_rsa -C "legacy-access"
```

Права доступа (File Permissions)

В Linux/Unix права доступа — это первый эшелон защиты. Неверные права часто приводят к отказу в доступе со стороны sshd (режим StrictModes), но более опасно, когда они слишком открыты.

Путь / Файл	Права (chmod)	Описание безопасности
~/ssh/	700 (drwx---)	Директория доступна только владельцу. Группа и остальные не видят даже список файлов.
id_ed25519	600 (-rw-----)	Критично. Приватный ключ. Чтение/запись только у владельца.
id_ed25519.pub	644 (-rw-r--r--)	Публичный ключ. Можно читать всем (он не секретен).
authorized_keys	600 (-rw-----)	Список доступа на сервере. Защищаем от записи, чтобы злоумышленник не добавил свой ключ.

Защита конфигураций WireGuard:

В контексте защиты ключей нельзя не упомянуть и другие секреты, часто хранящиеся на тех же машинах, например, VPN-конфигурации. Файлы WireGuard (.conf), содержащие Private Key, должны защищаться так же строго, как и SSH-ключи.

- Конфиги должны лежать в /etc/wireguard/ (владелец root).
- Права доступа строго **600**.
- **Важно:** Удаляйте файлы .conf из папок «Загрузки» или «Документы» после настройки. Оставленный в пользовательской папке конфиг — подарок для malware.

Passphrase и физическая безопасность

Нужен ли пароль на ключ? Да. Хранение ключа без кодовой фразы (passphrase) допустимо только в строго контролируемых автоматизированных средах (CI/CD), где используются другие методы компенсации рисков.

На рабочей станции инженера ключ **обязан** быть зашифрован.

Однако пароль защищает только файл на диске. Здесь вступает в игру физическая безопасность:

- **Full Disk Encryption (LUKS):** Если диск ноутбука не зашифрован, злоумышленник при краже устройства просто смонтирует файловую систему и скопирует ваши ключи. Пароль на SSH-ключе замедлит его, но не остановит (возможен оффлайн брутфорс). Шифрование диска — обязательное требование.
- **SSH-agent:** Для удобства мы используем агент, который хранит расшифрованный ключ в оперативной памяти. Это удобно, но создает риск. Злоумышленник с правами root или физическим доступом к разблокированной машине может сделать дамп памяти процесса агента и извлечь ключи.

Меры защиты агента:

- Ограничение времени жизни ключа в памяти: `ssh-add -t 3600` (удалить через час).
- Использование аппаратных токенов (YubiKey, HSM), где приватный ключ физически невозможно извлечь.

4. Ландшафт угроз: Актуальные уязвимости

Даже при идеальной настройке существуют векторы атак на сам протокол и его реализации.

TOFU (Trust On First Use) и MITM

Классическая проблема SSH. При первом подключении клиент не знает сервер и спрашивает пользователя: *"ED25519 key fingerprint is SHA256:.... Are you sure?"*. Пользователи почти всегда нажимают "Yes" не глядя. В этот момент атакующий (Man-In-The-Middle) может подставить свой ключ.

Решение: Использование SSH-сертификатов, подписанных доверенным центром (CA), полностью устраняет необходимость доверять неизвестным хостам.

CVE-2025-26465: Атака через DNS-верификацию

Недавняя уязвимость, связанная с опцией `VerifyHostKeyDNS`. Она позволяет клиенту проверять отпечаток ключа сервера через DNS (записи SSHFP).

- *Суть атаки:* MITM-злоумышленник может манипулировать DNS-ответом, вызывая состояние нехватки памяти (OOM) или обход валидации на стороне клиента.
- *Статус:* Уязвимость была активна на FreeBSD (2013–2023) и других системах, где эта опция была включена. Требуется немедленного патчинга клиента.

Terrapin Attack (CVE-2023-48795)

Первая практически реализуемая атака на целостность SSH-рукопожатия. Атакующий может «обрезать» (truncate) пакеты в начале соединения, заставляя стороны договориться о менее безопасных алгоритмах или отключить расширения безопасности, при этом подписи сессии останутся валидными.

Защита: Обновление OpenSSH и отключение уязвимых шифров (ChaCha20-Poly1305 в старых реализациях был подвержен, сейчас исправлен через "strict KEX").

XZ Utils Backdoor (CVE-2024-3094): Кошмар цепочки поставок

Инцидент, который потряс мир ИБ в 2024 году. В библиотеку сжатия liblzma, используемую процессом sshd, был внедрен бэкдор. Злоумышленник под псевдонимом "Jia Tan" в течение двух лет внедрял код, который активировался только в процессе сборки RPM/DEB пакетов. Бэкдор перехватывал функцию проверки RSA-ключей, позволяя атакующему со своим «мастер-ключом» получить RCE (Remote Code Execution) на сервере без аутентификации, обходя все логи. Это напоминание о том, что даже Open Source не гарантирует безопасности, если Supply Chain скомпрометирована.

5. Управление ключами в Enterprise-среде

Когда количество серверов переваливает за сотню, управление ключами превращается в хаос. **Типичные проблемы:**

- **Зомби-ключи:** Сотрудник уволился, его учетную запись в AD закрыли, но его публичный ключ остался в `authorized_keys` на десятке серверов под пользователем `root` или `deploy`.
- **Дубликаты:** Использование одной пары ключей для доступа к Prod, Dev и Test средам. Компрометация ключа на тестовом стенде открывает доступ к продакшену.

Стратегия ротации: Регулярная смена ключей обязательна для соответствия стандартам (PCI DSS, ISO 27001, приказы ОАС).

- **High-risk (Root, технические учетки):** Ротация каждые 30–90 дней.
- **Medium-risk (DevOps, администраторы):** Каждые 90–180 дней.
- **Low-risk:** До 1 года.

Автоматизация: Используйте Ansible/Puppet для массового обновления файла `authorized_keys`. Ручное управление на масштабе невозможно.

6. SSH Сертификаты: Будущее уже здесь

Лучший способ решить проблемы статических ключей — отказаться от них в пользу **SSH Certificates**. В этой модели:

- Организация создает свой Certificate Authority (CA).
- Публичный ключ CA прописывается на всех серверах (TrustedUserCAKeys).
- Пользователь не копирует свой ключ на серверы. Он подписывает свой публичный ключ у CA (через внутренний портал авторизации) и получает сертификат.
- Сертификат живет короткое время (например, 8 часов — один рабочий день).

Преимущества:

- Нет необходимости чистить `authorized_keys` при увольнении — сертификат просто истечет.
- В сертификат вшиты права: можно разрешить пользователю только определенные команды или доступ только к конкретным хостам.
- Полная защита от TOFU.

7. Hardening и Compliance (Требования ОАС/ТЗИ)

Для прохождения аттестации системы защиты информации (СЗИ) сервер SSH должен быть жестко сконфигурирован. Приводим эталонный пример конфигурации `/etc/ssh/sshd_config` с пояснениями.

1. Отключаем пароли. Только ключи.

`PasswordAuthentication no`

`ChallengeResponseAuthentication no`

`PubkeyAuthentication yes`

2. Запрещаем вход root. Администрирование только через sudo с персональных учеток.

`PermitRootLogin no`

3. Ограничиваем список пользователей (Белый список)

`AllowUsers admin_user deploy_user@10.0.0.0/24`

4. Отключаем X11 Forwarding.

Это предотвращает атаку, когда скомпрометированный сервер может перехватывать нажатия клавиш на клиенте через проброшенный графический интерфейс.

`X11Forwarding no`

5. Ограничение алгоритмов (Crypto Policy)

Оставляем только стойкие шифры, отключаем слабые MAC.

`HostKey /etc/ssh/ssh_host_ed25519_key`

`KexAlgorithms sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-sha256`

`Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com`

`MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com`

```
# 6. Журналирование (Обязательно для OAC)
SyslogFacility AUTH
LogLevel VERBOSE
```

Мониторинг и аудит: Просто писать логи недостаточно — их нужно читать. Для соответствия требованиям регуляторов настройте отправку логов в SIEM. Ищите аномалии:

- Входы в ночное время.
- Множественные ошибки Preauth (признак сканирования или брутфорса).
- Использование команды sudo сразу после входа.

Для проверки текущей стойкости сервера используйте утилиту ssh-audit:

```
ssh-audit 192.168.1.10
```

Она покажет, поддерживаются ли устаревшие алгоритмы обмена ключами (Diffie-Hellman group1) или слабые MAC (MD5, SHA1), которые необходимо отключить.

SSH-ключи — это фундамент безопасности инфраструктуры. Их компрометация равносильна передаче ключей от офиса грабителям. Профессиональный подход к безопасности SSH требует перехода от парадигмы «настроил и забыл» к непрерывному циклу управления:

- **Генерация:** Только Ed25519 с защитой паролем.
- **Хранение:** Шифрование диска (LUKS) и правильные права доступа (600).
- **Архитектура:** Переход на SSH-сертификаты в крупных средах.
- **Контроль:** Регулярный аудит конфигурации sshd и мониторинг журналов.

Только комплексный подход, учитывающий физическую безопасность, криптографическую стойкость и процедурную дисциплину, позволит вашей системе успешно пройти аттестацию и противостоять современным киберугрозам.